COMP 3380 W26

Final Project Report

Formula 1 Database System

Members:

Duc Cam Thai [thaidc]

Duy Anh Ta [tada]

Syed Affan Akhtar [akhtarsa]

1- Introduction

This project was very in-depth and involved several steps. First, we retrieved public data and then we designed, implemented, and interacted with a relational database. The data we used for this project was historical data for each race in the history of Formula 1 from 1950 to 2024 and allows users to run queries on the dataset through a command-line interface.

The project was multi-step, beginning with the design of a database model, followed by the implementation of the database on the department server and finally the development of an interface that would allow people to look at the information as well as pose their own queries about the data. The system was written to run off the University's Aviary servers and connects to a SQL Server hosted on Uranium. The interface allows for one to view the information about each race, as well as pose questions about individual drivers, certain races, the timing and frequency of pit stops, the current season standings, and other information about the history of Formula 1.

2- Summary of the Dataset

The dataset that was used for this project was the Formula 1 World Championship dataset covering the years 1950-2024. This data was sourced from Kaggle and contains historical race, driver, team, circuit and results data.

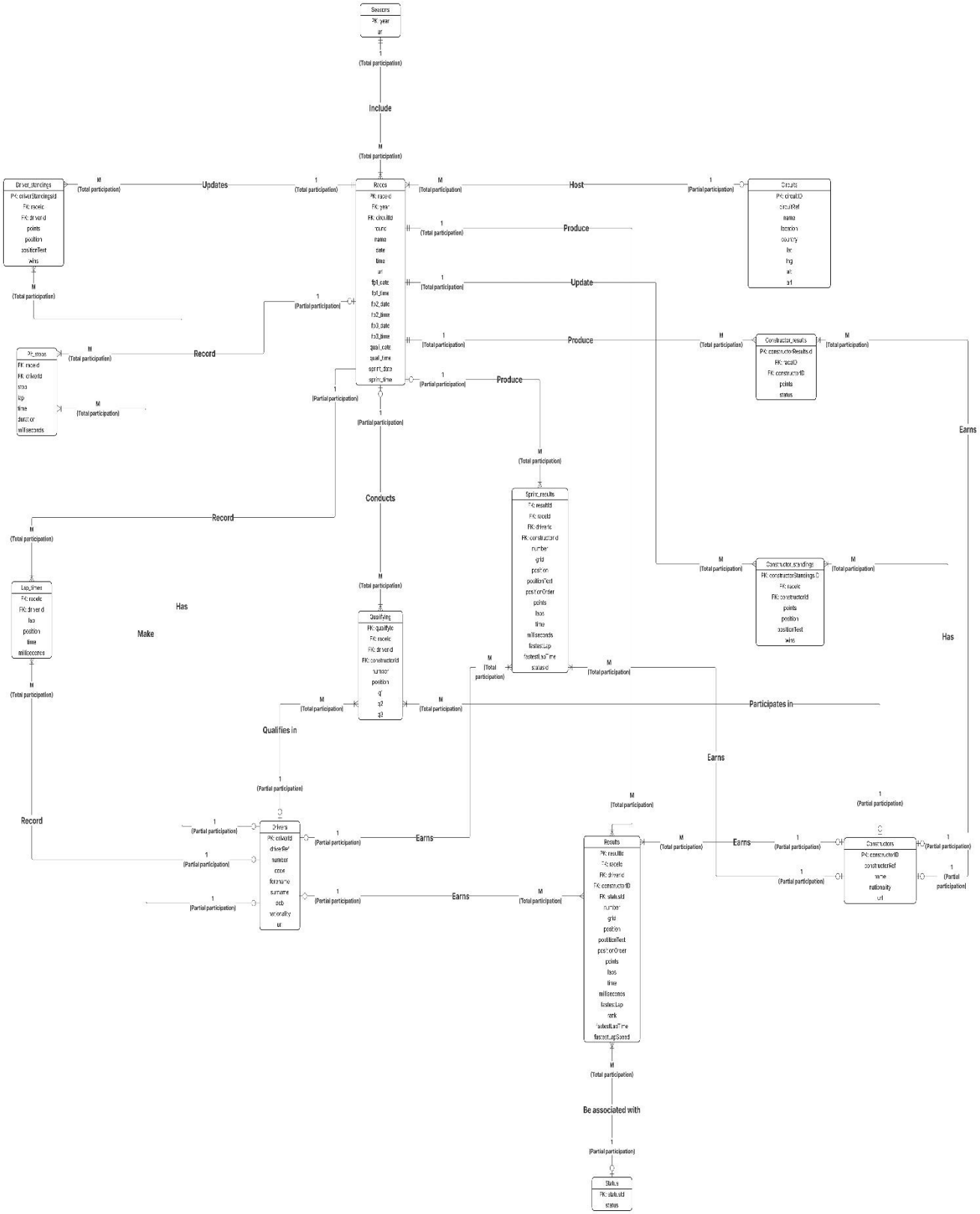
All of the data comes in the form of tables, and we have covered the drivers, the teams (constructors), individual races, seasons, the result of the race, the lap times and the pit stops. There is also a table with the qualifying results and a table with the overall championship positions at the end of the season.

We chose Formula 1 data for this dataset because there are many natural relationships between the individual rows of data. In a typical season a driver will complete 20 plus races. Each race will be held at a specific race track, or circuit. Each race will have two or more constructors competing at the race and each constructor will have one or more drivers. Over the course of a season, each driver will amass a certain number of points with the driver amassing the most points declared the championship champion at the end of the season.

The dataset used in this project has 14 tables and over 700,000 records, which makes it large enough to be thoroughly tested. Because of the many connections (driverId, raceId, constructorId, circuitId etc.), it is not a simple wheel-and-spoke data model but a highly interconnected relational database model.

Before we could use the data in the database, we had some preprocessing to undergo. The data was downloaded from Kaggle in a spreadsheet-like format of tables. These tables were converted into CSV files, and then into SQL insert statements to make loading the data into our SQL Server database easier.

ER Diagram


Seasons
PK: year
ur
Include
Driver_standings
PK: driverStandingsId
FK: raceId
FK: driverId
points
position
positionText
wins
Updates
Races
PK: raceId
FK: year
FK: circuitId
round
name
date
time
ur
fp1_date
fp1_time
fp2_date
fp2_time
fp3_date
fp3_time
quali_date
quali_time
sprint_date
sprint_time
Host
Circuits
PK: circuitID
circuitRef
name
location
country
lat
lng
alt
ur1
Produce
Update
Pit_stops
FK: raceId
FK: driverId
stop
lap
time
duration
milliseconds
Record
Constructor_results
PK: constructorResultsId
FK: raceID
FK: constructorID
points
status
Earns
Sprint_results
PK: resultId
FK: raceId
FK: driverId
FK: constructorId
number
grid
position
positionText
positionOrder
points
laps
time
milliseconds
fastestLap
fastestLapTime
statusId
Conducts
Lap_times
FK: raceId
FK: driverId
lap
position
time
milliseconds
Has
Make
Qualifying
FK: qualifyId
FK: raceId
FK: driverId
FK: constructorId
number
position
q1
q2
q3
Constructor_standings
PK: constructorStandingsID
FK: raceId
FK: constructorId
points
position
positionText
wins
Has
Participates in
Qualifies in
Earns
Drivers
PK: driverId
driverRef
number
code
forename
surname
dob
nationality
ur
Results
PK: resultId
FK: raceId
FK: driverId
FK: constructorID
FK: statusId
number
grid
position
positionText
positionOrder
points
laps
time
milliseconds
fastestLap
rank
fastestLapTime
fastestLapSpeed
Constructors
PK: constructorID
constructorRef
name
nationality
url
Be associated with
Status
PK: statusId
status
(Total participation)
(Partial participation)
M
1

This ER diagram illustrates the main entities within the dataset and their corresponding links. The main entities within the dataset are: Drivers, Constructors, Races, Circuits, Seasons, and Results. Supporting tables are also illustrated including Lap Times, Pit Stops, Qualifying and Standings.

They are connected through foreign keys. For example, A driver participates in many races, A race occurs at a circuit, A constructor fields drivers in races, A race produces results and standings. Several tables can be selected for analysis, beyond the typical two-table approach, due to the highly interconnected nature of the database.

3- Data Model Discussion

When creating the relational model, we used tables to represent the structure of Formula 1 Racing. Redundancy in the data was also eliminated. The data was organised into tables, each organised around a concept. For example, information about drivers (name, nationality etc.) was put into a “Drivers” table, and then the results of each race were associated with the appropriate driver, so that driver information did not need to be repeated in every race table. Information about each racing circuit was put into a “Circuits” table, rather than repeating it in every table for races that took place on that circuit.

One of the components of the data model that was a bit tricky to figure out how to store were the results of individual races and the final season standings. The results of a race are dependent on which drivers actually competed in that particular race (i.e., it cannot just be a list of every driver with their standings for the race). We chose to set up the results table as a many-to-many relationship between drivers and races (as a race could have multiple results, and each result could be for a different constructor team).

The data also includes lap times and pit stops per race. These tables have a large number of rows per race and this has to be carefully modelled in order to establish the correct connections to the races and drivers’ tables.

Our database design followed the ER diagram quite closely. There are mostly many-to-one relationships. For example, there are many races in a single season and many results in a single race.

If the project were repeated, this data model would likely still be used because it captures the structure of the sport quite accurately while remaining flexible enough to support analytical queries.

4- Database Implementation

Our database itself was implemented using Microsoft SQL Server, which is the database system available on the department server Uranium. It was created using SQL scripts that defined the structure and dependencies of the tables in the database.

The Java program connects to the database using JDBC. The connection is established through a connection string that specifies the Uranium server address

and the database name. In the implementation, the connection is created using **DriverManager.getConnection(),** which authenticates the user and establishes communication between the Java application and the SQL Server database. Once the connection is established, SQL queries can be executed using prepared statements, and the results are returned to the interface for display.

In addition, another important feature of the implementation is that the interface includes functionality to delete all data and repopulate the database. The repopulation process reads SQL insert statements from files and executes them in batches, allowing the database to be reconstructed automatically. This feature completes one of the project requirements that involved clearing and re-populating the databases when needed.

5- Interface Description

The front-end interface of the project is a command line program written in Java; the program runs on the Aviary servers and communicates with the Uranium database server.

The program begins by asking the university username and password to authenticate the connection to the SQL Server database.

```
[akhtarsa@pelican ~/COMP-3380-Formula-1-Database-Project]> make run
javac -cp .:lib/mssql-jdbc-13.4.0.jre11.jar:src src/*.java
java -cp .:lib/mssql-jdbc-13.4.0.jre11.jar:src SQLServer
Enter username: akhtarsa
Enter password:

Enter an option to run a query.
Enter 'h' for help.
```

After authentication, the user is presented with a command menu that lists all available queries.

```
Option: h

Available commands:
 h   -> show help
 q   -> quit

Query commands:
 1   -> Driver performance by season
 2   -> Constructors with most DNFs by season
 3   -> Hat-trick list
 4   -> Grand Slam list
 5   -> Lap degradation
 6   -> Most championship points in a season
 7   -> Most consecutive driver wins
 8   -> Drivers with most DNFs per season
 9   -> Most wins but not champions
 10  -> Races for a single constructor where no drivers are classified
 11  -> Races where all drivers are classified
 12  -> Wins without pole
 13  -> Worst pole-to-win ratio
 14  -> Fastest average pit stop team by season
 15  -> Constructor teammate head-to-head
 16  -> List drivers
 17  -> List constructors
 18  -> List circuits
 19  -> Delete all data
 20  -> Repopulate database
```

Users can interact by specifying the number corresponding to the question of their choice, at which time the program will issue the appropriate SQL query and the display the results in a tabular format that is simple to read and understand.

```
Option: 16
Drivers List
id          forename        surname
----------------------------------------
1           Lewis           Hamilton
2           Nick            Heidfeld
3           Nico            Rosberg
4           Fernando        Alonso
5           Heikki          Kovalainen
6           Kazuki          Nakajima
7           Sébastien       Bourdais
8           Kimi            Räikkönen
9           Robert          Kubica
10          Timo            Glock
11          Takuma          Sato
12          Nelson          Piquet Jr.
13          Felipe          Massa
14          David           Coulthard
15          Jarno           Trulli
16          Adrian          Sutil
17          Mark            Webber
18          Jenson          Button
19          Anthony         Davidson
20          Sebastian       Vettel
```

We organized the results in a clear format with column headings and consistent layout to improve readability. The program also contains several functions that will make your daily work much easier, more user-friendly and safer:

- A help command that displays all available queries
- Checks to ensure program does not crash if it is given an invalid option.
- Prepared statements, which ensure that user input cannot be used for SQL injection.

6- Interesting Queries

We have made significant strides in enabling our program to support a large number of “queries” that an analyst/grader with F1 data would ask, In fact, we have implemented the ability to support over 15 different types of analytical queries including many that require aggregation (e.g., average speed by team), grouping (e.g. driver average speed by track), or even multi-table joins.

There were four queries implemented in this project that were particularly quite complex to implement.

1- **Grand Slam Achievements:** one of the more challenging parts of this project to get right has been the ability to query for Grand Slam by starting from the pole, winning the race, setting the fastest race lap and leading every lap of the race. This query combines several tables in a single query, like race results, qualifying results, lap times and drivers. It then checks if this driver is also the

car that achieved the fastest qualifying time and held the fastest time for every lap in the race and compares this to the total number of laps in the race.

2- **Hat-Trick Achievements:** In the context of F1, a Hat-Trick would be for a driver to achieve all three, securing pole position for the race, setting the fastest lap of the race and winning the race. This query uses data from three tables, the qualifying table, results and the races table. It first lists all the drivers who won a race, and then it checks if that driver also started from the pole position for the race, setting the fastest lap of the race, and winning the race.
3- **Longest Consecutive Driver Win Streaks:** Finding the longest sequences of consecutive races won by drivers gives insight into historically dominant drivers and how they had to fight off other drivers to win the same race multiple times within a season. It first analyses the results of the races in chronological order, groups all consecutive wins together to form a streak, calculates the length of the streaks and returns the longest streaks found.
4- **Lap Time Degradation Between Pit Stops:** This query calculates the difference in lap time between two parts of the race separated by a pitstop. This query can be used to analyse tire wear and to determine ideal race strategy. In F1, lap times tend to increase as tires deteriorate over the course of the race. This query compares lap times at the start of the race to lap times at the end of the race to determine performance loss. We used both pit stop and lap time data and tables. For each driver, we find the laps between each pair of stops and calculate the difference of the lap times. We then average the degradation.

7- Conclusion

The Formula 1 data set we used has many relations between the different tables, a relational database management system was therefore a good choice. Because the data contains structure and relationships between drivers, races, teams and results, it would be best to put the data into a relational database format. This would then allow us to write SQL queries to join together tables to extract the data we need.

Other database systems could also be used. A graph database would be more interesting to index to relationships between drivers, teams, and races in a more flexible way.

This database could also be useful as a teaching tool for future instances of the course. The database is large enough that interesting questions can be asked of it, and it is structured in such a way that most people should be able to understand the relationships between the different types of entities in the database. This project reinforced many of the key concepts already covered in this course. It also involved building a full-fledged database system from scratch, from data modelling and database implementation to application.

Group Contributions

Duc Cam Thai:

- Database interface implementation
- Java query implementation
- Data processing and import
- Database population scripts

Duy Anh Ta:

- ER diagram design and implementation
- Relational model development
- SQL query design
- Data processing and import

Syed Affan Akhtar:

- Database connection, testing and verification
- SQL query assistance
- Final Project Report
- Query testing